UrbanSphere Project 2

Focused on applying intermediate SQL techniques.

Venkata Sai Varun Vedula

31 July 2025, 3:10 PM

1 Introduction:

Urban Sphere is a simulated U.S. omnichannel retail database created to replicate real-world sales, customer, and product data for analytics practice.
Project 2 takes this dataset and applies SQL to solve practical business problems that a retail analyst, business analyst, or business intelligence professional might face in day-to-day work.

The project is designed to mimic the workflow of an analyst:

1. Start with data exploration and profiling
2. Move into targeted analysis for products, customers, and regions
3. Derive actionable insights using SQL

2 Goal of the Project:

The primary objectives of this project are to:

1. Strengthen SQL query writing skills through real-world business use cases.
2. Build a portfolio-ready project that demonstrates technical proficiency and analytical thinking.
3. Generate actionable retail insights using clean, reproducible SQL queries.
4. Showcase competence in SQL concepts relevant for Business Analyst, Data Analyst, and BI roles.

3 Project Overview and Key SQL Concepts Covered

The project contains 20 business questions based on Urban Sphere's dataset, covering customer retention, product performance, regional revenue trends, and sales forecasting.
From these, 15 key questions were selected for the final report due to their high business value and strong demonstration of SQL concepts.

Key SQL Concepts Covered:

- Subqueries – For filtering, comparisons, and anti-join logic.
- Joins (Left, Inner, Anti) – To combine data across multiple tables.

- Aggregate Functions – For KPI calculations like total revenue, quantity sold, and average spend.
- Window Functions – Ranking, value lookups, and aggregations (running & rolling totals).
- Rolling and Running Totals – For time-series trend analysis.
- CASE Statements – For creating business-friendly categories and segmentation.

4 About the database

Table Name	**Description**
customers	Customer profile including ID, name, segment, region ID, and join date.
regions	List of geographic regions where customers are located.
products	Product catalog with product name, category, price, and stock details.
orders	Records of each order including customer ID, product ID, quantity, and order date.
payments	Payment records for each order including final amount and payment status.

4.1 ERD

Entities:

1. Customers → linked to Regions
2. Customers → place Orders
3. Orders → contain Products
4. Orders → have Payments

Relationships:

1. customers.region_id → regions.region_id
2. orders.customer_id → customers.customer_id
3. orders.product_id → products.product_id
4. payments.order_id → orders.order_id

5 SQL USE CASES

1 What is the total number of unique customers who placed an order in the last calendar year?

```
USE UrbanSphere;

SELECT * FROM customers;
SELECT * FROM orders;

SELECT distinct
    COUNT(customer_id) AS unique_customers
FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

Purpose: Measures the size of the active customer base in the past year to evaluate engagement levels. Achieved by filtering orders table by date range and using COUNT(DISTINCT customer_id) to get unique counts.

unique_customers
103

2 Which product category generated the highest total revenue in the last six months?

```
SELECT * FROM products;
SELECT * FROM orders;
SELECT * FROM payments;
SELECT MAX(order_date) FROM orders;

SELECT
    p.category,
    SUM(pa.final_amount) as total_revenue
FROM products as p
JOIN orders as o
ON p.product_id = o.product_id
JOIN payments as pa
ON o.order_id = pa.order_id
WHERE order_date >= DATE_SUB('2025-06-24', INTERVAL 6 MONTH)
GROUP BY p.category
ORDER BY SUM(pa.final_amount) DESC LIMIT 1;
```

Purpose: Identifies recent top-performing categories for prioritizing inventory and promotions. Accomplished by joining products, orders, and payments, applying a 6-month date filter, aggregating with SUM(final_amount), and sorting to find the top result.

category	total_revenue
Fashion	1604.40

3 List all customers who have not placed a single order in the last three months.

```sql
SELECT * FROM orders;
SELECT * FROM customers;

SELECT
    c.customer_id,
    c.customer_name
FROM customers as c
LEFT JOIN orders as o
ON c.customer_id = o.customer_id
AND o.order_date >= DATE_SUB('2025-06-24', INTERVAL 3 MONTH)
WHERE o.order_id IS NULL;
```

Purpose: Helps target dormant customers for reactivation campaigns. Implemented using an anti-join (LEFT JOIN with IS NULL) between customers and recent orders filtered by a 3-month date range.

customer_id	customer_name
101	Clayton Yates
102	Kristi Hays
105	William Kim
106	Dr. Douglas Glover
107	Bethany Bowen
108	Nicholas Mcpherson
110	Jillian Morse
112	Zachary Riley
113	Samuel Leonard
114	Lauren Price
115	Douglas Robinson
116	Dr. Charles Hunt
120	Brenda Beck
124	Michael Martin
126	Margaret Rogers
127	Deborah Brewer
130	Jared Miles
132	Annette Graham
133	Jeremiah Sanders
134	Denise Sandoval

4 For each customer, show their most recent order date along with the total number of orders they have placed so far.

```sql
SELECT
    c.customer_name,
    c.customer_id,
    COUNT(o.order_id) AS total_orders,
    MAX(o.order_date) AS most_recent_order
FROM orders as o
LEFT JOIN customers as c
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY c.customer_id ASC;
```

Purpose: Combines recency and frequency to assess customer lifecycle stage. Executed by joining customers with orders, grouping by customer, and using MAX(order_date) for recency and COUNT(order_id) for frequency.

customer_name	customer_id	total_orders	most_recent_order
Clayton Yates	101	4	2024-10-20
Kristi Hays	102	2	2025-03-14
Aaron Lewis	103	3	2025-05-01
Ms. Monica Taylor	104	2	2025-05-03
William Kim	105	9	2025-02-21
Bethany Bowen	107	8	2025-03-11
Nicholas Mcpherson	108	7	2025-03-21
Robert Whitehead	109	6	2025-05-20
Jillian Morse	110	2	2024-10-15
Timothy Logan	111	5	2025-06-24
Zachary Riley	112	5	2024-12-06
Samuel Leonard	113	2	2025-01-17
Lauren Price	114	3	2024-11-30
Douglas Robinson	115	2	2024-09-06
Dr. Charles Hunt	116	4	2025-02-09
Denise Dickerson	117	5	2025-03-28
Nathan Hill	118	6	2025-05-12
Brian Todd	119	3	2025-03-24
Brenda Beck	120	4	2024-09-11
Theresa Johnson	121	3	2025-03-29
Jose Hall	122	8	2025-04-09
Amber Henderson	123	3	2025-06-05
Michael Martin	124	2	2025-01-01
Tammy Meyer	125	3	2025-04-10
Margaret Rogers	126	4	2025-01-17
Deborah Brewer	127	4	2025-02-16
Samuel Schultz	128	5	2025-06-18
Andrew Newton	129	6	2025-04-20
Jared Miles	130	6	2024-09-19
Brianna Lewis	131	6	2025-06-21
Annette Graham	132	4	2024-09-06
Jeremiah Sanders	133	3	2025-01-29
Denise Sandoval	134	7	2024-11-27
Carlos Munoz	135	4	2025-05-10

5 Which products have never been sold, and what is their category?

```sql
SELECT * FROM products;
SELECT * FROM orders;

SELECT
    p.product_id,
    p.product_name,
    p.category
FROM products AS p
LEFT JOIN orders AS o
ON p.product_id = o.product_id
WHERE o.order_id IS NULL;
```

Purpose: Flags underperforming SKUs for potential delisting or promotional focus. Solved using an anti-join (LEFT JOIN with IS NULL) between products and orders to find items with no matching sales records.

(Note: All products have been sold at least once, indicating that none of them have never been sold.)

product_id	product_name	category

6 For each region, what is the total revenue generated in the last year?

```sql
SELECT * FROM regions;
SELECT * FROM customers;
SELECT * FROM payments;
SELECT * FROM orders;

SELECT
    r.region_name,
    SUM(pa.final_amount) AS total_revenue_2024
FROM payments AS pa
LEFT JOIN orders as o
ON pa.order_id = o.order_id
LEFT JOIN customers AS c
ON c.customer_id = o.customer_id
LEFT JOIN regions AS r
ON c.region_id = r.region_id
WHERE o.order_date >= '2024-01-01' AND o.order_date <= '2024-12-31'
GROUP BY r.region_name
ORDER BY SUM(pa.final_amount) DESC;
```

Purpose: Compares regional performance for budget allocation and sales strategy. Implemented by joining regions, customers, orders, and payments, filtering by year, and aggregating revenue with SUM(final_amount) grouped by region.

region_name	total_revenue_2024
Pacific Northwest	3086.79
West Coast	2647.33
Midwest	2627.95
East Coast	2190.24
South	1928.09

7 Show the top 3 customers (by total spend) for each region.

```sql
SELECT
customer_name,
region_name,
total_spend
FROM (
    SELECT
    c.customer_name,
    r.region_name,
    SUM(pa.final_amount) AS total_spend,
    ROW_NUMBER() OVER (PARTITION BY r.region_name ORDER BY SUM(pa.final_amount) DESC) AS rn
    FROM customers AS c
    JOIN orders as o
    ON c.customer_id = o.customer_id
    JOIN regions as r
    ON c.region_id = r.region_id
    JOIN payments AS pa
    ON o.order_id = pa.order_id
    GROUP BY c.customer_id, c.customer_name, r.region_name)t
WHERE rn <= 3
ORDER BY region_name, total_spend DESC;
```

Purpose: Highlights high-value accounts for relationship management. Achieved by joining region, customer, order, and payment data, grouping by customer within each region, and applying ROW_NUMBER() OVER (PARTITION BY region ORDER BY SUM(final_amount) DESC) to rank and filter the top 3 per region.

customer_name	region_name	total_spend
Andrew Newton	East Coast	868.06
Zachary Riley	East Coast	684.19
Brenda Beck	East Coast	563.24
Jose Hall	Midwest	1000.99
Bethany Bowen	Midwest	870.97
Margaret Rogers	Midwest	619.17
William Kim	Pacific Northwest	1246.48
Denise Sandoval	Pacific Northwest	1183.30
Nathan Hill	Pacific Northwest	640.96
Carlos Munoz	South	765.17
Denise Dickerson	South	639.88
Lauren Price	South	491.32
Nicholas Mcpherson	West Coast	992.40
Brianna Lewis	West Coast	739.54
Samuel Schultz	West Coast	646.33

8 Which customers have never purchased an Electronics product?

```sql
SELECT * FROM customers;
SELECT * FROM orders;
SELECT * FROM products;

SELECT
c.customer_id,
c.customer_name
FROM customers c
WHERE c.customer_id NOT IN (
    SELECT o.customer_id
    FROM orders o
    JOIN products p
    ON o.product_id = p.product_id
    WHERE p.category = 'Electronics'
);
```

Purpose: Identifies cross-sell opportunities to grow category penetration. Solved using a subquery with a NOT IN clause to exclude customers found in joined orders–products records where category = 'Electronics'.

customer_id	customer_name
102	Kristi Hays
106	Dr. Douglas Glover
109	Robert Whitehead
112	Zachary Riley
115	Douglas Robinson
117	Denise Dickerson
119	Brian Todd
124	Michael Martin
135	Carlos Munoz

9 For each month in the past year, display the total number of orders and the running total of orders up to that month.

```sql
SELECT
month,
total_orders,
SUM(total_orders) OVER (ORDER BY month) AS running_total
FROM (
    SELECT
        MONTH(order_date) AS month,
        SUM(quantity) AS total_orders
    FROM orders
    WHERE order_date <= '2024-12-31'
    GROUP BY MONTH(order_date)
) t
ORDER BY month;
```

Purpose: Tracks seasonality and cumulative growth over time. Implemented by grouping orders by month using MONTH(order_date), summing quantities, and applying SUM(total_orders) OVER (ORDER BY month) for the running total.

month	total_orders	running_total
1	17	17
2	10	27
3	28	55
4	15	70
5	10	80
6	19	99
7	16	115
8	16	131
9	35	166
10	15	181
11	18	199
12	10	209

10 For each customer, calculate the average time gap (in days) between their consecutive orders.

```
SELECT
customer_id,
AVG(days_gap) AS avg_days_between_orders
FROM (
    SELECT
    customer_id,
    DATEDIFF(order_date, LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date ASC)) AS days_gap
    FROM orders
) t
WHERE days_gap IS NOT NULL
GROUP BY customer_id;
```

Purpose: Measures purchase cadence to improve retention and replenishment timing. Achieved with LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) to get prior order dates, DATEDIFF to calculate gaps, and AVG(days_gap) to find the average per customer.

customer_id	avg_days_between_orders
101	77.6667
102	54.0000
103	207.0000
104	426.0000
105	47.0000
107	60.0000
108	67.1667
109	91.8000
110	158.0000
111	90.7500
112	81.5000
113	226.0000
114	64.5000
115	93.0000
116	108.3333
117	89.5000
118	98.8000
119	159.5000
120	64.3333
121	132.0000
122	54.5714
123	100.0000
124	14.0000
125	222.5000
126	71.3333
127	105.3333
128	38.7500
129	57.4000
130	47.0000
131	96.8000
132	77.3333
133	110.5000
134	53.3333
135	98.0000

11 For each product, show the total quantity sold in each month and a rolling 3-month total of quantity sold for that product.

```sql
SELECT
    product_name,
    total_quantity,
    SUM(total_quantity) OVER (
        PARTITION BY product_name
        ORDER BY order_year, order_month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS rolling_3_month_total
FROM (
    SELECT
        p.product_name,
        YEAR(o.order_date) AS order_year,
        MONTH(o.order_date) AS order_month,
        SUM(o.quantity) AS total_quantity
    FROM products AS p
    LEFT JOIN orders AS o ON p.product_id = o.product_id
    WHERE o.order_date >= '2024-01-01' AND o.order_date <= '2024-12-31'
    GROUP BY p.product_name, YEAR(o.order_date), MONTH(o.order_date)
) t
ORDER BY product_name, order_year, order_month;
```

Purpose: Smooths sales fluctuations to identify demand trends for planning. Implemented by aggregating monthly totals per product and applying SUM(total_quantity) OVER (PARTITION BY product_name ORDER BY year, month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) for the rolling sum.

product_name	total_quantity	rolling_3_month_total
Ago Set	1	1
Ago Set	3	4
Ago Set	1	5
Agreement XL	3	3
Agreement XL	1	4
Agreement XL	1	5
Agreement XL	2	4
Agreement XL	3	6
All Set	3	3
All Set	7	10
All Set	1	11
Almost Set	3	3
Almost Set	2	5
Almost Set	3	8
Begin Lite	1	1
Begin Lite	3	4
Begin Lite	2	6
Begin Lite	3	8
Begin Lite	2	7
Better Plus	5	5
Better Plus	3	8
Better Plus	2	10
Blue Pro	3	3
Blue Pro	1	4
Brother Max	3	3
Brother Max	6	9
Brother Max	2	11

12 Display each order’s date and the running total revenue generated across all orders up to that date.

```
SELECT
    o.order_date,
    SUM(p.final_amount) AS daily_revenue,
    SUM(SUM(p.final_amount)) OVER (ORDER BY o.order_date) AS running_total  #SUM(daily_revenue) written as SUM(SUM(p.final_amount))
FROM orders AS o
LEFT JOIN payments AS p ON o.order_id = p.order_id
GROUP BY o.order_date
ORDER BY o.order_date;
```

Purpose: Monitors sales growth trajectory over time. Executed by grouping by order_date, summing final_amount for daily revenue, and applying SUM(daily_revenue) OVER (ORDER BY order_date) for the cumulative total.

order_date	daily_revenue	running_total
2024-01-04	78.73	78.73
2024-01-12	231.87	310.60
2024-01-15	181.84	492.44
2024-01-16	81.45	573.89
2024-01-18	31.45	605.34
2024-01-21	129.54	734.88
2024-01-28	NULL	734.88
2024-01-30	171.19	906.07
2024-02-11	111.71	1017.78
2024-02-12	242.73	1260.51
2024-02-16	101.46	1361.97
2024-02-21	102.50	1464.47
2024-02-23	123.86	1588.33
2024-02-24	NULL	1588.33
2024-03-01	66.82	1655.15
2024-03-02	198.53	1853.68
2024-03-03	30.04	1883.72
2024-03-04	32.05	1915.77
2024-03-07	211.37	2127.14
2024-03-10	214.36	2341.50
2024-03-11	86.10	2427.60
2024-03-13	222.60	2650.20
2024-03-15	132.23	2782.43
2024-03-18	301.18	3083.61

13 Rank all products by total revenue generated, showing the product name, total revenue, and their overall rank.

```
SELECT
    p.product_name,
    SUM(pa.final_amount) AS total_revenue,
    RANK() OVER(ORDER BY SUM(pa.final_amount) DESC)
FROM products AS p
LEFT JOIN orders AS o
ON p.product_id = o.product_id
LEFT JOIN payments as pa
ON o.order_id = pa.order_id
GROUP BY p.product_name
ORDER BY total_revenue DESC;
```

Purpose: Establishes a product performance leaderboard for strategic merchandising. Achieved by joining products, orders, and payments, grouping by product, summing revenue, and applying RANK() OVER (ORDER BY total_revenue DESC).

product_name	total_revenue	RANK() OVER(ORDER BY SUM(pa.final_amount) DESC)
Brother Max	1577.28	1
Require Lite	1525.03	2
Since Max	1204.18	3
All Set	1075.07	4
Chair Max	1068.24	5
Purpose Set	1053.35	6
Agreement XL	976.90	7
Site Pro	937.16	8
Early Pro	848.84	9
Play Plus	795.50	10
Dog Plus	753.71	11
Offer Max	735.43	12
Sit Pro	720.86	13
Election Set	719.21	14
Face Plus	597.65	15
Begin Lite	531.47	16
Better Plus	491.49	17
Wait Set	436.21	18
Blue Pro	430.07	19
Performance Pro	362.11	20
Knowledge Lite	337.97	21
Ago Set	256.35	22
Mouth Lite	251.43	23
Discover Lite	207.46	24
Almost Set	149.52	25

14 For each customer, display their name and the date of their most recent order (latest order date).

```sql
SELECT
    c.customer_name,
    MAX(o.order_date) AS most_recent_order
FROM customers AS c
LEFT JOIN orders AS o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Purpose: Measures recency to identify loyal customers and those at churn risk. Implemented with a join between customers and orders, grouping by customer, and using MAX(order_date).

customer_name	most_recent_order
Clayton Yates	2024-10-20
Kristi Hays	2025-03-14
Aaron Lewis	2025-05-01
Ms. Monica Taylor	2025-05-03
William Kim	2025-02-21
Dr. Douglas Glover	NULL
Bethany Bowen	2025-03-11
Nicholas Mcpherson	2025-03-21
Robert Whitehead	2025-05-20
Jillian Morse	2024-10-15
Timothy Logan	2025-06-24
Zachary Riley	2024-12-06
Samuel Leonard	2025-01-17
Lauren Price	2024-11-30
Douglas Robinson	2024-09-06
Dr. Charles Hunt	2025-02-09
Denise Dickerson	2025-03-28
Nathan Hill	2025-05-12
Brian Todd	2025-03-24
Brenda Beck	2024-09-11

15 For each order, show the order ID, customer name, and the average order value of all orders placed by that customer

```
SELECT
    c.customer_name,
    o.order_id,
    p.final_amount,
    AVG(p.final_amount) OVER(PARTITION BY c.customer_id) AS avg_order
FROM orders AS o
JOIN customers AS c
ON o.customer_id = c.customer_id
JOIN payments AS p
ON o.order_id = p.order_id;
```

Purpose: Compares individual orders against customer averages to detect anomalies or VIP purchases. Achieved by joining customers, orders, and payments, and applying AVG(final_amount) OVER (PARTITION BY customer_id).

customer_name	order_id	final_amount	avg_order
Clayton Yates	309	158.85	109.765000
Clayton Yates	321	66.82	109.765000
Clayton Yates	355	99.48	109.765000
Clayton Yates	378	113.91	109.765000
Kristi Hays	404	49.02	72.420000
Kristi Hays	414	95.82	72.420000
Aaron Lewis	322	222.60	140.470000
Aaron Lewis	341	37.21	140.470000
Aaron Lewis	383	161.60	140.470000
Ms. Monica Taylor	336	30.04	69.420000
Ms. Monica Taylor	337	108.80	69.420000
William Kim	305	58.16	155.810000
William Kim	315	174.13	155.810000
William Kim	339	214.36	155.810000
William Kim	373	23.20	155.810000
William Kim	393	227.12	155.810000
William Kim	396	201.34	155.810000
William Kim	409	111.71	155.810000
William Kim	415	236.46	155.810000
William Kim	443	NULL	155.810000
Bethany Bowen	327	182.25	145.161667
Bethany Bowen	328	76.06	145.161667
Bethany Bowen	331	81.45	145.161667
Bethany Bowen	345	NULL	145.161667
Bethany Bowen	359	244.93	145.161667
Bethany Bowen	386	NULL	145.161667
Bethany Bowen	436	134.40	145.161667
Bethany Bowen	440	151.88	145.161667
Nicholas Mcpherson	304	168.96	141.771429

16 For each customer, classify them as ‘High Value’, ‘Medium Value’, or ‘Low Value’ based on their total spend (e.g., High Value: >$500, Medium: $200–$500, Low: <$200).

```
SELECT * FROM customers;
SELECT * FROM payments;
SELECT * FROM orders;

SELECT
    c.customer_name,
    SUM(p.final_amount) AS total_spend,
    CASE
    WHEN SUM(p.final_amount) > 500 THEN 'High Value'
    WHEN SUM(p.final_amount) BETWEEN 200 AND 500 THEN 'MEDIUM VALUE'
    WHEN SUM(p.final_amount) <= 200 THEN 'LOW VALUE'
    END
FROM customers AS c
JOIN orders AS o
ON c.customer_id = o.customer_id
JOIN payments AS p
ON o.order_id = p.order_id
GROUP BY c.customer_name
ORDER BY total_spend DESC;
```

Purpose: Segments customers into tiers for loyalty and marketing strategies. Implemented by aggregating spend per customer and applying a CASE statement to assign categories.

customer_name	total_spend	ValueLoaded
William Kim	1246.48	High Value
Denise Sandoval	1183.30	High Value
Jose Hall	1000.99	High Value
Nicholas Mcpherson	992.40	High Value
Bethany Bowen	870.97	High Value
Andrew Newton	868.06	High Value
Carlos Munoz	765.17	High Value
Brianna Lewis	739.54	High Value
Zachary Riley	684.19	High Value
Samuel Schultz	646.33	High Value
Nathan Hill	640.96	High Value
Denise Dickerson	639.88	High Value
Robert Whitehead	639.42	High Value
Margaret Rogers	619.17	High Value
Brenda Beck	563.24	High Value
Dr. Charles Hunt	528.01	High Value
Timothy Logan	497.47	MEDIUM VALUE
Lauren Price	491.32	MEDIUM VALUE
Clayton Yates	439.06	MEDIUM VALUE
Tammy Meyer	425.46	MEDIUM VALUE
Aaron Lewis	421.41	MEDIUM VALUE
Amber Henderson	421.07	MEDIUM VALUE
Jared Miles	365.98	MEDIUM VALUE
Deborah Brewer	307.23	MEDIUM VALUE
Douglas Robinson	303.74	MEDIUM VALUE
Samuel Leonard	274.53	MEDIUM VALUE
Jeremiah Sanders	262.54	MEDIUM VALUE
Jillian Morse	258.64	MEDIUM VALUE
Brian Todd	241.16	MEDIUM VALUE
Michael Martin	240.40	MEDIUM VALUE
Annette Graham	180.69	LOW VALUE
Kristi Hays	144.84	LOW VALUE
Ms. Monica Taylor	138.84	LOW VALUE
Theresa Johnson	NULL	NULL

17 For each product, show the product name and a flag called 'is_popular' which is 'Yes' if the product has been sold more than 20 times, else 'No'.

```sql
SELECT * FROM products;
SELECT * FROM orders;

SELECT
    p.product_name,
    SUM(o.quantity) total_quantity,
    CASE
    WHEN SUM(o.quantity) >= 20 THEN 'Popular'
    ELSE 'No'
    END
FROM products AS p
JOIN orders as o
ON p.product_id = o.product_id
GROUP BY p.product_name
ORDER BY total_quantity DESC;
```

Purpose: Identifies high-demand products for promotion and inventory focus. Implemented by summing quantities per product and applying a CASE statement to flag popular items.

product_name	total_quantity	FlagNote
Brother Max	25	Popular
Since Max	23	Popular
Purpose Set	21	Popular
Chair Max	19	No
Agreement XL	18	No
Site Pro	16	No
Early Pro	15	No
Require Lite	14	No
Play Plus	14	No
Election Set	13	No
Begin Lite	13	No
All Set	13	No
Offer Max	12	No
Sit Pro	10	No
Better Plus	10	No
Face Plus	9	No
Dog Plus	9	No
Wait Set	9	No
Performance Pro	9	No
Almost Set	9	No
Blue Pro	7	No
Ago Set	6	No
Mouth Lite	6	No
Discover Lite	4	No
Knowledge Lite	2	No

6 Key Takeaways:

1. Practiced intermediate SQL concepts such as rolling totals, running totals, ranking, and lag functions in a retail analytics context.
2. Built multi-table JOIN queries across five related tables (customers, regions, orders, products, payments) to extract insights reflecting realistic business operations.
3. Applied aggregate and window functions to analyze sales trends, segment customers, and rank products by revenue.
4. Gained confidence in combining temporal analysis with grouping and partitioning for monthly, daily, and category-level insights.
5. Developed subqueries and anti-joins to identify dormant customers, unsold products, and category gaps.
6. Strengthened understanding of CASE statements to create value tiers for customers and popularity flags for products.
7. Performed robust data filtering, date-based analysis, and NULL handling to ensure accurate and consistent outputs.
8. Produced well-documented, actionable insights directly linked to stakeholder needs, such as retention targeting and inventory prioritization.

7 Workflow Breakdown:

1. Reviewed the UrbanSphere schema from Project 1, focusing on table relationships between customers, products, orders, payments, and regions.
2. Selected 17 practical business use cases covering subqueries, joins, aggregates, window functions, rolling/running totals, and CASE statements.
3. Designed and developed queries to align with each business question, applying appropriate SQL concepts for accuracy.
4. Tested and validated each query incrementally, refining filters, joins, and groupings.
5. Applied advanced functions including ranking, value lookups, cumulative calculations, subqueries, and anti-joins for targeted exclusions.
6. Integrated business context into each query by explaining the technical method and its relevance to decision-making.

7. Documented all use cases with references, purposes, and function details for report readiness.
8. Compiled the final report with Key Takeaways, Workflow Breakdown, and Conclusion, ensuring consistent presentation with Project 1.

8 Conclusion:

The UrbanSphere Project 2 built on the foundation of Project 1 by expanding into intermediate-level SQL techniques applied to realistic retail business scenarios. Throughout 17 well-defined use cases, the project demonstrated proficiency in translating business requirements into technical solutions using subqueries, multi-table joins, aggregate functions, window functions, rolling and running totals, and CASE statements. Each query was paired with a clear business purpose, ensuring the outputs were actionable and aligned with stakeholder needs. The structured workflow—from schema review and question selection to query development, testing, and final documentation—ensured complete coverage of the target SQL concepts while maintaining business relevance. This project reinforces readiness for analyst roles where the ability to derive insights from data, communicate results effectively, and apply a range of SQL techniques is critical.